Halo2 통합

개요

Scroll ZKEVM은 Halo2 영지식 증명 백엔드를 사용하여 모든 회로를 구현합니다. Halo2는 PLONK 증명 시스템을 기반으로 하며, 재귀적 증명과 효율적인 검증을 지원합니다. 이 문서는 ZKEVM 구현체에서 Halo2 API를 어떻게 사용하는지 설명합니다.

Halo2 API 개요

핵심 컴포넌트

Halo2는 다음과 같은 핵심 컴포넌트를 제공합니다:

1. **Circuit 트레이트**: 회로의 구조와 제약 조건을 정의
2. **ConstraintSystem**: 제약 조건을 등록하고 관리
3. **Layouter**: Witness 데이터를 회로에 할당
4. **Challenge API**: 검증자의 무작위 값(challenge)을 생성하고 사용
5. **Column 타입**: Advice, Fixed, Instance 컬럼 정의

주요 임포트

```
use halo2_proofs::{
circuit::{Layouter, SimpleFloorPlanner, Value},
plonk::{Circuit, ConstraintSystem, Error, Expression, Column, Advice, Fixed},
```

```
};
```

Circuit 트레이트 구현

Circuit 트레이트 정의

모든 회로는 `Circuit` 트레이트를 구현해야 합니다:

```rust
pub trait Circuit<F: Field> {
type Config;
type FloorPlanner;
type Params;

fn without_witnesses(&self) -> Self;
fn configure(meta: &mut ConstraintSystem<F>) -> Self::Config;
fn synthesize(
&self,
config: Self::Config,
layouter: impl Layouter<F>,
) -> Result<(), Error>;
}
```

SuperCircuit 구현 예시

파일: `zkevm-circuits/src/super_circuit.rs`

```
impl<
const MAX_TXS: usize,
const MAX_CALLDATA: usize,
const MAX_INNER_BLOCKS: usize,
const MOCK_RANDOMNESS: u64,
> Circuit<Fr> for SuperCircuit<Fr, MAX_TXS, MAX_CALLDATA, MAX_INNER_BLOCKS, MOCK
{
type Config = (SuperCircuitConfig<Fr>, Challenges);
type FloorPlanner = SimpleFloorPlanner;
type Params = ();


fn  without_witnesses(&self) -> Self {
let  dummy_block = Block {
circuits_params: self.circuit_params,
..Default::default()
};
Self::new_from_block(&dummy_block)
}


fn  configure(meta: &mut ConstraintSystem<Fr>) -> Self::Config {
let  challenges = Challenges::construct(meta);
(
```

```
SuperCircuitConfig::new(
meta,
SuperCircuitConfigArgs {
max_txs: MAX_TXS,
max_calldata: MAX_CALLDATA,
max_inner_blocks: MAX_INNER_BLOCKS,
mock_randomness: MOCK_RANDOMNESS,
challenges,
},
),
challenges,
)
}


fn synthesize(
&self,
(config, challenges): Self::Config,
mut layouter: impl Layouter<Fr>,
) -> Result<(), Error> {
let challenges = challenges.values(&layouter);


config.u8_table.load(&mut layouter)?;
config.u16_table.load(&mut layouter)?;
```

```
self.synthesize_sub(&config, &challenges, &mut layouter)
}
}
```

주요 메서드 설명

1. `configure`

회로의 구조를 정의하는 단계입니다. 이 메서드에서:

- 컬럼(Advice, Fixed, Instance)을 생성
- 제약 조건(constraints)을 등록
- 룩업 테이블을 설정
- Challenge를 할당

```
fn configure(meta: &mut ConstraintSystem<Fr>) -> Self::Config {
// Challenge 생성
let challenges = Challenges::construct(meta);
// 테이블 생성
let tx_table = TxTable::construct(meta);
let rw_table = RwTable::construct(meta);
// 회로 설정 생성
SuperCircuitConfig::new(meta, SuperCircuitConfigArgs {
max_txs: MAX_TXS,
challenges,
```

```
// ...
})
}
```

2. synthesize

Witness 데이터를 회로에 할당하는 단계입니다:

- Challenge 값을 가져옴
- Fixed 테이블을 로드
- 각 서브 회로의 `synthesize_sub` 호출

```
fn synthesize(
&self,
(config, challenges): Self::Config,
mut layouter: impl Layouter<Fr>,
) -> Result<(), Error> {
// Challenge 값 가져오기
let challenges = challenges.values(&layouter);

// Fixed 테이블 로드
config.u8_table.load(&mut layouter)?;
// 서브 회로 할당
self.synthesize_sub(&config, &challenges, &mut layouter)
```

```
}
```

3. `without_witnesses`

증명 키 생성 시 사용되는 더미 회로를 생성합니다:

```
fn without_witnesses(&self) -> Self {
let dummy_block = Block {
circuits_params: self.circuit_params,
..Default::default()
};
Self::new_from_block(&dummy_block)
}
```

SubCircuit 패턴

ZKEVM은 `SubCircuit` 트레이트를 정의하여 회로 간 일관된 인터페이스를 제공합니다.

SubCircuit 트레이트

파일: `zkevm-circuits/src/util.rs`

```
pub trait SubCircuit<F: Field> {
type Config: SubCircuitConfig<F>;

fn unusable_rows() -> usize {
```

```
256
}

fn new_from_block(block: &witness::Block) -> Self;

fn instance(&self) -> Vec<Vec<F>> {
vec![]
}

fn synthesize_sub(
&self,
config: &Self::Config,
challenges: &Challenges<Value<F>>,
layouter: &mut impl Layouter<F>,
) -> Result<(), Error>;

fn min_num_rows_block(block: &witness::Block) -> (usize, usize);
}
```

SubCircuitConfig 트레이트

```
pub trait SubCircuitConfig<F: Field> {
type ConfigArgs;
```

```
fn new(meta: &mut ConstraintSystem<F>, args: Self::ConfigArgs) -> Self;
}
```

사용 예시

각 회로는 `SubCircuit` 을 구현합니다:

```
impl SubCircuit<Fr> for EvmCircuit<Fr> {
type Config = EvmCircuitConfig<Fr>;

fn new_from_block(block: &Block) -> Self {
// 블록 데이터로부터 회로 생성
}

fn synthesize_sub(
&self,
config: &Self::Config,
challenges: &Challenges<Value<Fr>>,
layouter: &mut impl Layouter<Fr>,
) -> Result<(), Error> {
// Witness 할당 로직
}
```

```
fn min_num_rows_block(block: &Block) -> (usize, usize) {
// 필요한 행 수 계산
}
}
```

Layouter 사용

Layouter 개념

`Layouter` 는 Witness 데이터를 회로의 컬럼에 할당하는 인터페이스입니다. Halo2는 `SimpleFloorPlanner` 를 기본 레이아웃 전략으로 사용합니다.

Region 할당

회로는 `Region` 을 통해 셀에 값을 할당합니다:

```
layouter.assign_region(
|| "region name",
|mut region| {
// Advice 컬럼에 값 할당
let cell = region.assign_advice(
|| "cell name",
column,
offset,
|| Value::known(value),
)?;
```

```
// Fixed 컬럼에 값 할당
region.assign_fixed(
|| "fixed cell",
fixed_column,
offset,
|| Value::known(fixed_value),
)?;
Ok(cell)
},
)?;
```

테이블 로드

Fixed 테이블은 `load` 메서드로 초기화됩니다:

```
impl<F: Field> U8Table<F> {
pub fn load(&self, layouter: &mut impl Layouter<F>) -> Result<(), Error> {
layouter.assign_region(
|| "u8 table",
|mut region| {
for i in 0..256 {
region.assign_fixed(
|| format!("u8 value {}", i),
self.value,
```

```
i,
|| Value::known(F::from(i  as u64)),
)?;
}
Ok(())
},
)
}
}
```

서브 회로 할당 패턴

SuperCircuit은 각 서브 회로를 순차적으로 할당합니다:

```
fn  synthesize_sub(
&self,
config: &Self::Config,
challenges: &Challenges<Value<Fr>>,
layouter: &mut  impl Layouter<Fr>,
) -> Result<(), Error> {
// EVM Circuit 할당
self.evm_circuit
.synthesize_sub(&config.evm_circuit, challenges, layouter)?;
// Keccak Circuit 할당
```

```
self.keccak_circuit

.synthesize_sub(&config.keccak_circuit, challenges, layouter)?;

// State Circuit 할당

self.state_circuit

.synthesize_sub(&config.state_circuit, challenges, layouter)?;

// ... 다른 회로들

Ok(())

}
```

Challenge API

Challenge 개념

Challenge는 검증자가 제공하는 무작위 값으로, Fiat-Shamir 변환을 통해 비대화형 증명을 만드는 데 사용됩니다. ZKEVM은 여러 단계(phase)에서 challenge를 사용합니다.

Challenge 정의

파일: `zkevm-circuits/src/util.rs`

```
#[derive(Default, Clone, Copy, Debug)]

pub  struct Challenges<T = Challenge> {

evm_word: T,

keccak_input: T,

lookup_input: Option<T>,

}
```

Challenge 생성

Challenge는 `configure` 단계에서 생성됩니다:

```
impl Challenges {
pub fn construct<F: Field>(meta: &mut ConstraintSystem<F>) -> Self {
Self {
// FirstPhase 이후 사용 가능
evm_word: meta.challenge_usable_after(FirstPhase),
keccak_input: meta.challenge_usable_after(FirstPhase),
// SecondPhase 이후 사용 가능
lookup_input: Some(meta.challenge_usable_after(SecondPhase)),
}
}
}
```

Challenge를 Expression으로 변환

제약 조건에서 사용하기 위해 `Expression` 으로 변환:

```
pub fn exprs<F: Field>(&self, meta: &mut ConstraintSystem<F>) -> Challenges<Ex
let [evm_word, keccak_input] = query_expression(meta, |meta| {
[self.evm_word, self.keccak_input]
.map(|challenge| meta.query_challenge(challenge))
});
```

```
let lookup_input = self.lookup_input
.map(|c| query_expression(meta, |meta| meta.query_challenge(c)));
Challenges {
evm_word,
keccak_input,
lookup_input,
}
}
```

Challenge 값 가져오기

`synthesize` 단계에서 실제 challenge 값을 가져옵니다:

```
pub fn values<F: Field>(&self, layouter: &impl Layouter<F>) -> Challenges<Valu
Challenges {
evm_word: layouter.get_challenge(self.evm_word),
keccak_input: layouter.get_challenge(self.keccak_input),
lookup_input: self.lookup_input.map(|c| layouter.get_challenge(c)),
}
}
```

Challenge 사용 예시

RLC (Random Linear Combination)

Challenge는 주로 RLC를 통해 여러 바이트를 하나의 필드 원소로 압축하는 데 사용됩니다:

```
// 바이트 배열을 RLC로 인코딩
pub fn rlc_be_bytes<F: Field>(bytes: &[u8], rand: Value<F>) -> Value<F> {
rand.map(|rand| {
bytes.iter().fold(F::zero(), |acc, byte| {
acc * rand + F::from(*byte as u64)
})
})
}
```

제약 조건에서 사용

```
// Keccak 입력 RLC 계산
let input_rlc = bytes.iter().enumerate().fold(
0.expr(),
|acc, (idx, byte)| {
acc + byte.expr() * challenges.keccak_input_powers_of_randomness()[idx]
},
);
```

Phase 시스템

Halo2는 다단계(multi-phase) 증명을 지원합니다:

1. **FirstPhase**: 기본 witness 할당

2. **SecondPhase**: FirstPhase challenge를 사용한 witness 할당

3. **ThirdPhase**: SecondPhase challenge를 사용한 witness 할당 (SuperCircuit에서 사용)

```
// SecondPhase 컬럼 생성
let advice_column = meta.advice_column_in(SecondPhase);


// Challenge는 이전 phase 이후에 사용 가능
let challenge = meta.challenge_usable_after(FirstPhase);
```

ConstraintSystem 사용

제약 조건 등록

`configure` 단계에서 제약 조건을 등록합니다:

```
meta.create_gate("gate name", |meta| {
let selector = meta.query_selector(q_enable);
let a = meta.query_advice(col_a, Rotation::cur());
let b = meta.query_advice(col_b, Rotation::cur());
let c = meta.query_advice(col_c, Rotation::cur());
// a * b = c 제약 조건
vec![selector * (a * b - c)]
});
```

Lookup 등록

룩업 테이블을 사용한 제약 조건:

```
meta.lookup_any("lookup name", |meta| {
let selector = meta.query_selector(q_lookup);
let value = meta.query_advice(col_value, Rotation::cur());
vec![(selector * value, table_column)]
});
```

Rotation 사용

다른 행의 값을 참조할 때 `Rotation` 을 사용합니다:

```
let current = meta.query_advice(column, Rotation::cur()); // 현재 행
let prev = meta.query_advice(column, Rotation::prev()); // 이전 행
let next = meta.query_advice(column, Rotation::next()); // 다음 행
let offset = meta.query_advice(column, Rotation(5)); // 5행 뒤
```

Column 타입

Advice Column

Witness 데이터를 저장하는 컬럼:

```
let advice_col = meta.advice_column();
meta.enable_equality(advice_col); // Copy constraint 활성화
```

Fixed Column

회로 구조에 고정된 값을 저장:

```
let fixed_col = meta.fixed_column();
```

Instance Column

공개 입력(Public Input)을 저장:

```
let instance_col = meta.instance_column();
meta.enable_equality(instance_col);
```

Selector

특정 행에서 제약 조건을 활성화/비활성화:

```
let selector = meta.selector();


// 사용 시
meta.create_gate("gate", |meta| {
let s = meta.query_selector(selector);
// s가 1일 때만 제약 조건 적용
vec![s * constraint_expression]
});
```

회로 빌드 프로세스

1. 회로 생성

```
let circuit = SuperCircuit::new_from_block(&block);
```

2. 필요한 행 수 계산

```
let (_, rows_needed) = SuperCircuit::min_num_rows_block(&block);
let k = log2_ceil(SuperCircuit::unusable_rows() + rows_needed);
```

3. 증명 키 생성

```
let params = ParamsKZG::<Bn256>::new(k);
let vk = keygen_vk(&params, &circuit)?;
let pk = keygen_pk(&params, vk, &circuit)?;
```

4. 증명 생성

```
let mut transcript = Blake2bWrite::init(vec![]);
create_proof::<_, ProverGWC<_>, _, _, _, _>(
&params,
&pk,
&[circuit],
```

```
&[&instance],
OsRng,
&mut transcript,
)?;
let proof = transcript.finalize();
```

최적화 기법

SimpleFloorPlanner

ZKEVM은 `SimpleFloorPlanner` 를 사용하여 회로 레이아웃을 자동으로 관리합니다:

```
type FloorPlanner = SimpleFloorPlanner;
```

이는 회로 설계자가 셀 배치를 수동으로 관리할 필요 없이 Halo2가 자동으로 최적화된 레이아웃을 생성하도록 합니다.

Blinding Factors

Halo2는 보안을 위해 회로 끝에 블라인딩 행을 추가합니다:

```
fn unusable_rows() -> usize {
256 // blinding_factors + 1
}
```

병렬 증명 생성

여러 회로의 증명을 병렬로 생성할 수 있습니다 (aggregator에서 사용).

관련 문서

- 02-제약-조건-구축.md - ConstraintBuilder 패턴
- .../08-최적화-기법/02-RLC-기법.md - Challenge를 사용한 RLC
- .../01-아키텍처/01-전체-구조.md - 전체 시스템 아키텍처
- .../07-증명-집계/02-증명-생성.md - 증명 생성 프로세스

참고 자료

- Halo2 Book
- Halo2 API Documentation
- `zkevm-circuits/src/util.rs` - SubCircuit 트레이트 정의
- `zkevm-circuits/src/super_circuit.rs` - Circuit 트레이트 구현 예시